

Unit-3

Cohesion: Cohesion is a measure that defines degree of intra-dependability within elements of a module. The greater the cohesion, the better is the program design.

- Cohesion is a property of an individual module.
- It is the concept which tries to capture the relation between elements of the same module.
- Cohesion of a module represents how tightly bound the internal elements of the module are.
- Cohesion is a measure of the degree to which the elements of a module are functionally related.

Types of Cohesion:

- 1. Functional Cohesion:** Functional Cohesion is said to exist if the different elements of a module, cooperate to achieve a single function.
- 2. Sequential Cohesion:** A module is said to possess sequential cohesion if element of a module form the components of the sequence, where the output from one component of sequence is input to next.
- 3. Communicational Cohesion:** A module is said to have communicational cohesion, if all tasks of the module refer to or update same data structure, e.g., the set of functions defined on an array or a stack.
- 4. Procedural Cohesion:** A module is said to be procedural cohesion if particular sequence of steps

has to be carried out for achieving goal. e.g algorithm for decoding message.

5. Temporal Cohesion: When a module includes functions that are associated by the fact that all the methods must be executed in the same time, the module is said to exhibit temporal cohesion.

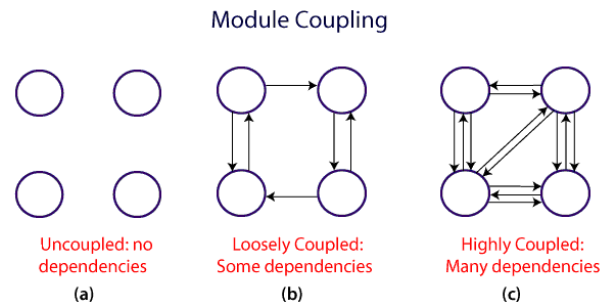
6. Logical Cohesion: A module is said to be logically cohesive if all the elements of the module perform a similar operation. For example Error handling.

7. Coincidental Cohesion: A module is said to have coincidental cohesion if it performs a set of tasks that are associated with each other very loosely.

Coupling:

- Coupling is a measure that defines the level of inter-dependability among modules of a program.
- It tells at what level the modules interfere and interact with each other.
- The lower the coupling, the better the program.
- It is the strength of interconnections between modules and it is the measure of the degree of interdependence between modules.
- Two modules with high coupling are strongly interconnected and thus, dependent on each other.

The various types of coupling techniques are shown in fig:



Types of Coupling:

1.Data Coupling:

- Two modules are data coupled if they communicate via an elementary data item which is passed as a parameter between the two.
- Data coupling is when two modules interact with each other by passing data (as parameter).

2.Stamp Coupling:

- Two modules are stamp coupled if they communicate using composite data items such as structure, objects, etc.
- For example, passing structure variable in C or object in C++ language to a module.

3.Control Coupling:

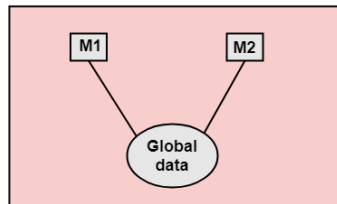
- When module A transfers control to module B by procedure call, it is called control coupling.
- Two modules are called control-coupled if one of them decides function of the other module or changes its flow of execution.

4.External Coupling:

- In external coupling, the modules depend on other modules, external to the software being developed or to a particular type of hardware.
- Ex- protocol, external file, device format, etc.

5.Common Coupling:

- Two modules are common coupled if they share same global data area or data structure.
- When multiple modules have read and write access to some global data, it is called common or global coupling.



6.Content Coupling:

- This is the least desirable coupling.
- When a module can directly access or modify or refer to the content of another module, it is called content level coupling.
- This type of coupling occurs when one module modifies local data values or instructions in another module.

Coupling	Cohesion
Coupling is also called Inter-Module Binding.	Cohesion is also called Intra-Module Binding.
Coupling shows the relationships between modules.	Cohesion shows the relationship within the module.
Coupling shows the relative independence between the modules.	Cohesion shows the module's relative functional strength.
While creating, you should aim for low coupling, i.e., dependency among modules should be less.	While creating you should aim for high cohesion, i.e., a cohesive component/ module focuses on a single function (i.e., single-mindedness) with little interaction with other modules of the system.
In coupling, modules are linked to the other modules.	In cohesion, the module focuses on a single thing.

Function Oriented Design:

- In function-oriented design, the system comprises of many smaller sub-systems known as functions.
- These functions are capable of performing significant task in the system. The system is considered as top view of all functions.
- This design mechanism divides whole system into smaller functions, which provides means of abstraction by hiding the information and their operation.
- These functional modules can share information among themselves.

- Another characteristic of functions is that when a program calls a function, the function changes the state of the program.
- Function oriented design works well where the system state does not matter and program/functions work on input rather than on a state.

Design Process

- Function Oriented Design is an approach to software design where a system is looked upon as something that performs a set of functions.
- The system is decomposed into a set of interacting units where each unit has clearly defined function.
- The system is designed from functional viewpoint.
- Starting at the high level description of what the program does and then in each step, each function is successively refined into more detailed functions.

Function Oriented Design Strategies:

1.Data Flow Diagram (DFD): A data flow diagram (DFD) maps out the flow of information for any process or system. It uses defined symbols like rectangles, circles and arrows to show data inputs, outputs, storage points and the routes between each destination.

2.Data Dictionaries: Data dictionaries are simply repositories to store information about all data items

defined in DFDs. Data dictionaries include Name of the item, Aliases (Other names for items), Description / purpose, Related data items, Range of values.

3.Structure chart: It is the hierarchical representation of system which partitions the system into black boxes (functionality is known to users but inner details are unknown). Components are read from top to bottom and left to right. When a module calls another, it views the called module as black box, passing required parameters and receiving results.

4.Pseudocode: Pseudo Code is system description in short English like phrases describing the function. It uses keyword and indentation. Pseudo codes are used as replacement for flow charts. It decreases the amount of documentation required.

Disadvantages:

- 1.Writing pure functions causes reduction in the readability of the code.
- 2.Though writing pure functions is easy, combining the same with the rest of the application as well as the I/O operations is tough.
- 3.Writing programs in recursive style in place of using loops for the same can be a difficult task.

Advantages of object oriented over function oriented

- It models the real world very well.
- Programs are easy to understand and maintain.
- Offers code reusability. Already created classes can be reused without having to write them again.
- Facilitates the quick development of programs where parallel development of classes is possible.
- Programs are easier to test, manage and debug.

Object Oriented Design:

Object Oriented Design (OOD) works around the entities and their characteristics instead of functions involved in the software system.

- Object Oriented Design is an approach in which the system is looked upon as a collection of interacting objects.
- It focuses on the data that are to be manipulated by the program.
- Objects are independent entities.
- These objects have their attributes and behaviour.
- Each object maintains its own state and a defined set of operations which operate on that state.
- An individual object is an instance of a class.

Elements of Object-Oriented System

1. **Objects:** All entities involved in solution design are known as objects. For example, person, banks, company, and users are considered as objects. Every entity has some attributes associated with it and has some methods to perform on the attributes.
2. **Classes:** A class is a generalized description of an object. An object is an instance of a class. A class defines all attributes, which an object can have and methods, which represents functionality of object.
3. **Messages:** Objects communicate by message passing. Messages consist of the integrity of the target object, the name of the requested operation, and any other action needed to perform the function. Messages are often implemented as procedure or function calls.
4. **Abstraction :** Abstraction in OOPS is used to hide unnecessary information and display only necessary information to the users interacting. It is essential to represent real-world objects in a simplified manner for users to interact easily.
5. **Encapsulation:** The data and operations are linked to a single unit is called encapsulation. Encapsulation not only bundles essential information of an object together but also restricts access to data and methods from the outside world.

6. **Inheritance:** OOD allows similar classes to stack up in a hierarchical manner where the lower or sub-classes can import, implement, and re-use allowed variables and functions from their immediate superclasses. This property of OOD is called an inheritance. This makes it easier to create generalized classes from specific ones.
7. **Polymorphism:** OOD languages provide a mechanism where methods performing similar tasks but different in arguments, can be assigned same name. This is known as polymorphism, which allows a single interface is performing functions for different types. Depending upon type of arguments, the respective portion of the code gets executed.

Design Process:

Software design process can be perceived as series of well-defined steps. The design serves as an input for the development phase of software. In the object-oriented approach, the focus is on capturing the structure and behaviour of information systems into small modules that combines both data and process.

The main aim of Object Oriented Design (OOD) is to improve the quality and productivity of system analysis and design by making it more usable.

The various steps involved are-

A solution design is created from requirement or previous used system and/or system sequence diagram.

Objects are identified and grouped into classes on behalf of similarity in attribute characteristics.

Class hierarchy and relation among them is defined.

Application framework is defined.

Software Metrics:

- Software metrics is defined as “The continuous application of measurement based techniques to the software development process and its products to supply meaningful and timely management information together with use of those techniques to improve that process and its products”.
- Software metrics are measurement that improves process of developing software and its products.
- Software measures not only help to control the software development process but also aid to keep the quality of ultimate product excellent.
- Software metrics are measurement that improves all aspects of the management of that process.
- Software metric used during development of design provide quantitative checks on software design.

Categories of Software Metrics:

Product Metrics: These describe the characteristics of the product such as size, complexity, design features, performance, efficiency, reliability, portability etc.

Process Metrics: These describe the effectiveness and quality of processes that produce the software product.

Project Metrics: These describe project characteristics and execution.

Size Metrics: Lines of Code, calculated in thousands of delivered source code lines, denoted as KLOC.

Function Point: Function Point count defines the size of functional aspect of the software.

Complexity Metrics: Cyclomatic complexity quantifies the upper bound of number of independent paths in a program, which is perceived as complexity of the program or its modules. It is represented in terms of graph theory concepts by using control flow graph.

Quality Metrics: Defects, their types and causes, consequence, intensity of severity and their implications define the quality of the product.

The number of defects found in development process and number of defects reported by the client after the product is installed or delivered at client-end, define quality of the product.

Benefits of Software Metrics:

- Monitor and control the progress of development.
- Cost and size estimation.
- Determine relative complexity.
- Analyse the defects.
- Make better decisions.
- Improve visibility, planning and control
- Improve quality and productivity
- Discover more effective test technique.

Token Count:

- A program is considered to be series of tokens and if we count number of tokens, Tokens are classified as either operators or operands.
- Variables and constants are operands.
- Operators consist of arithmetic symbols such as +, -, . /, * and command names such as "while", "for", "printf", special symbols such as: =, braces, parentheses, and even function names.

The size of vocabulary of program which consists of number of unique tokens to build program is $n = n1 + n2$
n: vocabulary of a program.

n1: number of unique operators

n2: number of unique operands.

Halstead Theory of Software Science: According to Halstead, “A computer program is an implementation of an algorithm considered to be a collection of tokens which can be classified as either operators or operands”.

- Halstead software science measurements were introduced in 1977 by M.H. Halstead.
- Halstead Science is an estimation technique to find out size, time and effort of the software.
- Halstead developed complexity metrics to measure program complexity directly from source code.
- Halstead complexity is based upon counts of operators and operands.
- It is based on interpreting the source code as a sequence of tokens and classifying each token to be an operator or an operand.

Following table states parameters and meanings:

Parameter	Meaning	
n1	Number of unique operators	
n2	Number of unique operands	
N1	Number of total occurrence of operators	
N2	Number of total occurrence of operands	

Metric	Meaning	Mathematical Representation
n	Vocabulary	$n1 + n2$
N	Size	$N1 + N2$
V	Volume	$Length * \log_2 \text{Vocabulary}$
D	Difficulty	$(n1/2) * (N1/n2)$
E	Efforts	$Difficulty * Volume$

Program Volume (V) It is the actual size of a program if a uniform binary encoding for the vocabulary is used.

$$V = N * \log_2 n$$

Program Difficulty

The difficulty level (D) of the program is proportional to the number of the unique operator in the program.

$$D = (n_1/2) * (N_2/n_2)$$

Programming Effort (E)

The unit of measurement of E is elementary mental discriminations. $E = D * V$

Estimated Program Length

The length of a well-structured program is function of number of operators and operands. $N = N_1 + N_2$

Size of Vocabulary (n)

The size of vocabulary of a program, which consists of number of unique tokens used to build a program is:

$$n = n_1 + n_2$$

Advantages of Halstead's Theory:

- It does not require in-depth analysis of program.
- It predicts effort and time.
- It predicts rate of error.

- It is simple to calculate.
- It is useful in scheduling projects.
- It measures overall quality of programs.
- It can be used for any programming language.

Drawbacks of Halstead's Theory:

- It depends on usage of operators and operands in completed code.
- It has no use in predicting complexity of program at design level.

Design Metrics:

- Software metric used during development of design provide quantitative checks on software design.
- Measuring various design attributes is useful in predicting faults but also in comparing two designs.
- Design metrics are more suitable for estimation during the design phase.

The Commonly used Design Metrics are:

1. Architectural Design Metrics:

- Architectural or high level design metrics focus on characteristics of the program.
- These focus on architectural structure and the effectiveness of components in architecture.

It involves three types of software design complexity-

Structural complexity: includes two things: number of modules and interrelationships between these modules.

Data complexity: provides an indication of the complexity in the internal interface for a module.

System complexity: is defined as the sum of structural and data complexity.

2. Component Level Design Metrics:

- Component level design metrics focus on the internal characteristics of a software component.
- These measures quantify the quality of a component level design.
- These metrics include measures of 3 C's which are:

Cohesion: a collection of metrics that provide an indication of the cohesiveness of a module.

Coupling: Module coupling provides an indication of the “connectedness” of a module to other modules, global data, and the outside environment.

Complexity: Complexity metrics provide feedback during the software project to control design activity.

The most widely used complexity metric for computer software is **cyclomatic complexity:** The cyclomatic number is equal to the number of linearly independent paths through a program in its graphs representation.

Data Structure Metrics:

The set of metrics that measures the amount of data input to, processed in and output from software are called data structure metrics.

Some Data Structure Metrics are:

a.The amount of Data:

- Most compilers and assemblers generate a cross-reference list, indicating the line where a certain variable is declared and lines where it is referenced.
- One method for determining amount of data is to count the number of entries in cross-reference list.
- To measure amount of data other metric are:
 - **Number of Operands (η_2):** In this metric, the Number of operands used in program is counted.
 - **Total number of occurrence of variable (N_2):** In this total occurrence of variables are computed

b.The usage of Data within a Module: To compute intra-module data usage, the metrics – Live Variables and Variable Spans are used.

i) Live Variables:

- A variable is considered live from its first to last reference in a module where variable is referenced.
- The set of live variable for each statement can be computed easily by analysing the module code.

ii) Variable Span:

- Variable Span is the number of statements between two successive uses of a variable.
- Variable Span is considered a complexity measure, larger the span, more complex the module.

c.Program Weakness:

- The weakness of the module helps to estimate the testability and maintainability.
- If weakness of a module is more, testability will be greater and vice-versa.
- Weaker module will be more difficult to maintain.

Software Implementation:

- Software Implementation is the process of converting manual or old software with newly developed software and making it operational, without disturbing the functioning of organization.
- It covers the period from the acceptance of the tested design to its satisfactory operation.
- It is a major operation across whole organization structure and requires a great deal of planning.
- It requires a thorough knowledge of new software, its personnel need, hardware requirements, file and procedure for conversion activities.

Activities involved in software implementation:

1.Training Personnel: The activity of training the users is a necessary part of implementation of new software. It helps users in operating and maintaining the software.

2.Conversion: Conversion is the process of changing from the old software to the new software. When the new software is fully tested and proved, conversion from the old to the new is possible.

3.Post-Implementation Review: After software implementation and conversion, review of software is conducted by users and analysts. Post-Implementation review measures the software's performance against pre-defined requirements. It determines how well the software continues to meet performance specifications.

Various Types of Implementation:

1.Fresh Implementation:

- Implementation of totally new software by replacing manual system is called fresh implementation.
- The problems encountered are converting files, training users, creating accurate files and verifying printouts for integrity.

2.Replacement Implementation:

- Implementation of new software to replace existing old software is called replacement implementation.
- It is usually a different conversion.

3.Modified Implementation:

- Implementation of a modified application to replace an existing one, in the same software is called modified implementation.
- This type of conversion is relatively easy to handle, provided there can be no major changes in the files.

Relationship between Design and Implementation

- The relationship between design and implementation is crucial in software development. Design is the blueprint of the software system, while implementation is the process of turning that design into a functioning software system.
- Design provides the foundation for the software system, including its architecture, components, and user interfaces. It lays out the plan for how the software system will work and what it will look like. Implementation takes the design and creates a functional software system by writing code, testing it, and deploying it to a target environment.

- Design and implementation are interdependent and must be closely linked to ensure that the software system meets the business requirements and delivers the desired outcomes. A well-designed software system that is not properly implemented will not be effective, and a poorly designed software system will be difficult to implement and will not deliver the desired outcomes.

Implementation Issues:

1.Reuse:

- It refers to reusing existing components or systems.
- Component reuse implies that the components of an application from sub-systems may be reused.
- Object and function reuse implies that the software components that implement a single well defined object or function may be reused.

Reuse Problems:

- **Increased Maintenance Costs:** In case the source code of reused system is not available then maintenance cost may be increased.
- **Lack of tool support:** Tools may not support development with reuse. It may be difficult to integrate these tools with component system.

- **Understand old software:** Sometimes writing original software is seen as easier than reusing other's software.
- **Creating and maintaining a component library:** Creating a reusable component library and ensuring that software developer can use it can be expensive.
- **Finding, understanding and adapting reusable components:** Software components have to be discovered in a library, understood to work in a new environment.

2.Configuration Management:

- It refers to keeping track of different versions of software components being developed. It is a process of managing a changing software system.
- It enables the developers to:
 - Access project code, documents in controlled way.
 - Find out what changes have been made.
 - Compile and link components to create a system.
- Configuration management should describe:
 - Process for managing changes during development.
 - Ensure developers can access most recent version.
 - Maintain older version, support integration process.

Configuration Management Issues:

- **User Roles:** There are different kinds of users of system. So, different functionality requirements for configuration management may arise.
- **When to start using configuration management:** The point at which a project team should start using configuration management depends on capabilities of the system.
- **Control Level:** Different levels of control can be imposed over the product and its management. A number of procedures and tools together assist in carrying out the configuration management.

3.Host-target development:

- It refers to development versus production platform.
- Most software is developed on one computer (host) but runs on separate one (target). These are termed as development platform and execution platform.
- A platform is more than just hardware; it includes installed operating system and supporting software.

Host-target development issues:

- Development platform usually has different installed software than execution platform.
- These platforms have different architectures.

Coding:

- Coding is the process of transforming the design of the system during the design phase into a computer language format.
- Coding is the activity where a design is translated to a runnable code.
- Coding is the software activity where the detailed design specification is implemented as source code.
- The coding translates design of system produced during design phase into a programming language.
- The input to coding phase is the design document.
- During coding phase, different modules identified in the design document are coded according to the module specifications.
- The aim of coding is not only to produce a working system but to produce a good quality system.
- During coding phase, programmer eliminates all syntax and format errors from program and also all errors are detected and removed during this phase.

Need of Coding Standards:

- Good Coding standards enhance the code understanding and encourage good programming.
- To give a uniform appearance to the codes written by various programmers.

- To enhance code understanding etc.
- To make it easier for programmers to read each other's code i.e. it increases code understanding.
- To make it easier to maintain the code.

Goals of Coding: The main goal of the coding or programming is to convert design of system produced during design phase into a programming language.

- Making a program more readable. It will increase code understanding.
- To translate design of the system into a computer language format. The aim is to implement the design in the best possible way.
- To reduce the cost of later phases. The aim is not to simplify the job of the programmer but to simplify the job of the tester and the maintainer.
- A well written code reduces the testing and maintenance effort. Since testing maintenance cost of software are much higher than coding cost, the goal of coding is to reduce testing and maintenance effort.
- During coding the focus should be on developing programs that are easy to write. Simplicity and clarity should be aimed at while coding.

Coding Style:

- Code means the actual computer syntax or computer program used to perform computations.
- Coding Style refers to methods/rules of writing readable and understandable code to develop computer programs.
- It is an art of developing computer programs which can be written in different styles to solve a given problem.
- A coding style is the best which produces software that is:
 - Technically correct
 - Easily understandable
 - Maintainable
- A well-written program is more easily read and understood by both the developer and others who work with that program.
- Poor programming style makes program difficult to understand, hard to modify and impossible to maintain over a long period of time.

Rules for writing a Good Code / Programming Style:

1. Clarity and Simplicity of Expression: The programs should be designed in such a manner so that the objective of program and each statement in program is

crystal clear. The user has not to strain his mind in understanding the program.

2.Names: While writing a program, names should be meaningful. Names should be closely related to entity they represent and modules names should reflect their activity. This make code easier to read and maintain.

3.Control Constructs: Program starts from one defined point and execution terminates at one defined point. Thus program can be viewed as sequence of statements.

4.Gotos: Use of goto's should be avoided as much as possible. If a goto statement must be used, then forward transfer is acceptable rather than backward jump. Use of goto is acceptable if language does not have any other alternatives.

5.Information Hiding: Information hiding is an important style for a good programming language.

The principle of information hiding is that-the information captured in the data structures should be hidden from the rest of the system.

6.Module Interface: If a module has complex interface, it should be carefully examined. Such type of modules is not functionally cohesive and are implementing multiple functions. Thus any module which has complex interface must be broken into multiple modules.

7.Nesting: The control construct if-then-else can be nested. If the nesting becomes too deep, the programs become difficult to understand. Where possible try to avoid deep nesting in coding.

8.Module Size: The module size should be uniform. Size of module should not be too big or too small. If module size is too large it is not generally functionally cohesive. If the module size is too small, it leads to unnecessary overhead. Thus module size should be decided according to principle of cohesion and coupling.

9.Program Layout: Program should be organized in such a way so that it has more readability. It should be clear in the first attempt. Proper indentation, blank spaces, comments should be used to increase the readability of the programs.

10.Robustness: A robust program is one that produces either correct results or meaningful messages, irrespective of the fact how bad the input data may be. Many times, the input data fed to a program may be invalid and the program may not produce the correct results and result in abrupt termination of a program.

11.Internal Documentation: Internal Documentation helps to increase the readability and understanding. It should be given at appropriate places. It is useful when maintenance and editing is done.